Where Should a Robot Swarm See Its Neighbors? Communication-Range Transfer in a GNN Coverage Policy

Loïc Poisson **Harold Gardon** **Claire Fitussi**
Stanford University
{lpoisson, gardon21, cfitussi}@stanford.edu

1 Application domain

Problem. In *coverage control*, N robots must spread over an area so that their sensors watch the regions that matter, described by an *importance density field* (IDF) ϕ that the robots do not know in advance [Cortés et al., 2004]. Each robot only sees a 64×64 m sensor footprint and can only exchange messages with robots within its *communication range* R; there is no central computer. The communication network is a graph $G_R = (V, E_R)$ with one node per robot and an edge (i, j) whenever $\|p_i - p_j\| \leq R$. This makes decentralized control a natural graph learning problem: every robot must decide what to send to its neighbors and how to combine what it receives [Tolstaya et al., 2020].

Dataset. We use the **CoverageControl** library [KumarRobotics, 2024] (GPL-3.0, `pip install coverage_control`), released with the LPAC paper [Agarwal et al., 2025]. It provides (i) a simulator of 32 robots in a 1024×1024 m world, with IDFs drawn as mixtures of 32 random Gaussians; (ii) the published LPAC checkpoint and the 11 evaluation environments it ships with; and (iii) scripts that generate imitation data by running a *clairvoyant* centroidal-Voronoi (CVT) controller that knows ϕ and all positions. One training sample is the state of all 32 robots at one time step: each robot's four 32×32 input maps, its position, the communication graph, and the clairvoyant's velocity as target. Following the paper we generate 100,000 samples per training set. We also generated and saved 40 new evaluation environments. Everything we use is public; our generated environments, data-generation configurations and code will be released in our repository[1] before the final submission.

Task and metric. The learned policy maps each robot's local observations and messages to a velocity, i.e. node-level regression on G_R, trained by imitation (mean squared error to the clairvoyant). We evaluate it in closed loop for 600 steps and report the paper's metric: the coverage cost $J = \int \phi(q) \min_i \|q - p_i\|^2 \, dq$ averaged over the episode, divided by the clairvoyant's cost on the same environment (lower is better; centralized CVT scores about 2.5). Our protocol varies the *test-time* range $R \in \{128, 256, 512, 768, 1024\}$ m, reports means over environments with paired differences, and uses three training seeds to set noise thresholds.

Why this dataset. LPAC is a recent published GNN policy whose authors report an unexplained anomaly: performance degrades when R grows (their Sec. VI-H). The simulator lets us control graph density exactly, the model is written in PyTorch Geometric [Fey and Lenssen, 2019], and a full training run takes 2 h 35 on one Stanford FarmShare GPU, so retraining several variants is affordable.

[1] `https://github.com/loicpoisson/lpac-communication-range` (made public before the final submission).

CS224W: Machine Learning with Graphs, Stanford University, Autumn 2026. Project proposal (type 1: real-world application).

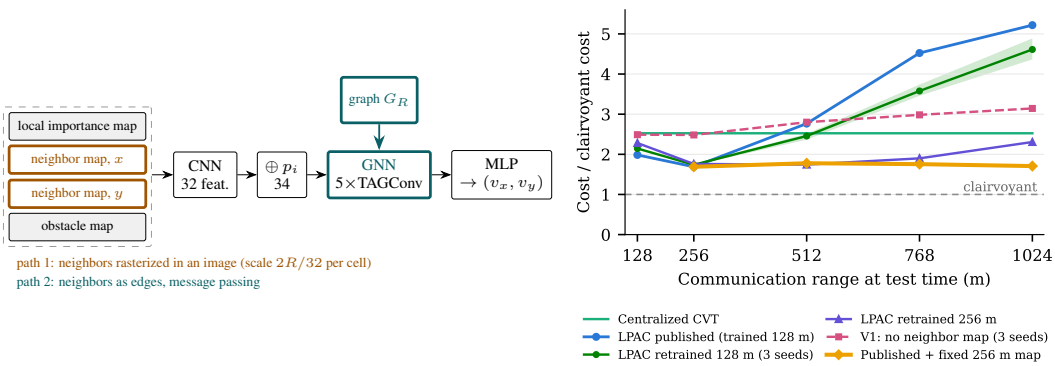


Figure 1: **Left:** LPAC on one robot. Neighbors enter twice: drawn in a 32×32 neighbor map read by the CNN (path 1) and as edges of G_R read by the GNN (path 2). **Right:** preliminary results, mean over the 11 released environments. Models trained at 128 m collapse when run with larger ranges; drawing the neighbor map at a fixed 256 m scale, without retraining, removes the collapse.

2 Graph ML techniques

Model: LPAC [Agarwal et al., 2025]. Each robot runs the same network (Fig. 1, left). A CNN encodes four 32×32 maps into 32 features, the robot position is appended, five TAGConv layers [Du et al., 2017] with $K = 3$ exchange information over G_R, and an MLP outputs the velocity. With node features X, adjacency A and degree matrix D,

$$X^{(\ell+1)} = \mathrm{ReLU}\Big(\textstyle\sum_{k=0}^{K} \big(D^{-1/2} A D^{-1/2}\big)^k X^{(\ell)} W_k^{(\ell)} \Big), \tag{1}$$

the GCN propagation [Kipf and Welling, 2017] applied up to 15 times in total. Two of the CNN's input channels form a *neighbor map*: robot j within range is written into cell $c_{ij} = \mathrm{round}\big((p_j - p_i)\frac{32}{2R} + 15.5\big)$ with value $(p_j - p_i)/R$, values summed per cell. GNNs suit this problem because message passing is exactly local communication: the policy is decentralized, permutation-equivariant, and independent of team size [Tolstaya et al., 2020].

Preliminary findings (motivate the project). (1) The published model (trained at 128 m) is the best controller at 128 and 256 m but its cost rises to 5.22 at 1024 m, worse than both CVT baselines, in all 11 environments; our three retrained models reproduce this (Fig. 1, right). (2) The GNN is not the culprit: at 1024 m, capping G_R to the 4 nearest neighbors leaves the cost at 5.11, while redrawing only the neighbor map at a fixed 256 m scale brings it to 1.71, the 256 m performance (paired difference $+0.02 \pm 0.16$); this holds on all our models and on 40 unseen environments. Effective rank [Roy and Vetterli, 2007] of the TAGConv outputs never collapses, ruling out over-smoothing. (3) Removing the neighbor map (V1) yields a flat but weaker policy (2.48 at 256 m vs 1.74).

Planned techniques. Our question is *where neighbor geometry should live*: in a rasterized image read by a CNN, or on the edges of the graph. We will compare, under identical data and training budgets: (a) inference-time interventions on each input path (graph capping, map rescaling); (b) LPAC trained on mixed communication ranges, a domain-randomization baseline [Tobin et al., 2017] (a first run avoids the collapse, 1.9 at 1024 m, but loses 0.4 at 256 m); (c) graph-native encoders that drop the neighbor map and carry the relative position $p_j - p_i$, normalized by a fixed length rather than by R, as edge attributes: GATv2 [Brody et al., 2022] (a first attempt underfits; we will tune it) and a PointNet-style edge convolution [Qi et al., 2017] placed before the TAGConv stack. Edge-based geometry is permutation-invariant and has no resolution, as in related swarm policies [Muthusamy et al., 2024, Owerko et al., 2025], but our first run shows it is not enough: the PointNet layer improves the map-free policy at the training range (2.21 vs 2.4–2.6) yet collapses even harder at 1024 m (5.8), because relative positions grow with R exactly like the map. We will therefore test range-invariant encodings: unit direction with a saturating distance, or geometry restricted to a fixed radius. Success: a variant within 0.05 of LPAC at 256 m that stays there up to 1024 m with no test-time fix (three seeds; released and new environments), which we will relate to degree-shift results for GNNs [Yehudai et al., 2021].

References

Saurav Agarwal, Ramya Muthukrishnan, Walker Gosrich, Vijay Kumar, and Alejandro Ribeiro. LPAC: Learnable perception-action-communication loops with applications to coverage control. *IEEE Transactions on Robotics*, 41, 2025. arXiv:2401.04855.

Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations*, 2022.

Jorge Cortés, Sonia Martínez, Timur Karataş, and Francesco Bullo. Coverage control for mobile sensing networks. *IEEE Transactions on Robotics and Automation*, 20(2):243–255, 2004.

Jian Du, Shanghang Zhang, Guanhang Wu, José M. F. Moura, and Soummya Kar. Topology adaptive graph convolutional networks. *arXiv preprint arXiv:1710.10370*, 2017.

Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.

KumarRobotics. CoverageControl: A library for coverage control with robot swarms, release v1.3.1. https://github.com/KumarRobotics/CoverageControl, 2024.

Shreyas Muthusamy, Damian Owerko, Charilaos I. Kanatsoulis, Saurav Agarwal, and Alejandro Ribeiro. Generalizability of graph neural networks for decentralized unlabeled motion planning. *arXiv preprint arXiv:2409.19829*, 2024.

Damian Owerko, Frederic Vatnsdal, Saurav Agarwal, Vijay Kumar, and Alejandro Ribeiro. MAST: Multi-agent spatial transformer for learning to collaborate. *arXiv preprint arXiv:2509.17195*, 2025.

Charles R. Qi, Li Yi, Hao Su, and Leonidas J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in Neural Information Processing Systems*, 2017.

Olivier Roy and Martin Vetterli. The effective rank: A measure of effective dimensionality. In *European Signal Processing Conference (EUSIPCO)*, 2007.

Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.

Ekaterina Tolstaya, Fernando Gama, James Paulos, George Pappas, Vijay Kumar, and Alejandro Ribeiro. Learning decentralized controllers for robot swarms with graph neural networks. In *Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pages 671–682, 2020.

Gilad Yehudai, Ethan Fetaya, Eli Meirom, Gal Chechik, and Haggai Maron. From local structures to size generalization in graph neural networks. In *International Conference on Machine Learning*, 2021.